

MLOps Assignment Installation Report

Project Title: Heart Disease Risk Prediction System

Repository: <https://github.com/Mathi27/end-to-end-cardio-pipeline>

Team: BITS Group 120 | MLOps

Team Details :

S.No	Name	Roll Number	Contribution %
1	Mathi Yuvarajan	2024AB05320	100 %
2	Tamilselvan	2024AA05346	100 %
3	Jaswanth Reddy Chagamreddy	2024aa05400	100 %
4	MANORANJAN DASH	2024ab05317	100 %
5	T Manoj Kumar Jankiram	2024AA05940	100 %

1. System Requirements

Minimum Requirements

OS: Linux/macOS/Windows (with WSL2)

CPU: 2 cores

RAM: 4GB

Storage: 10GB free space

Python: 3.9+

Docker: 20.10+

Minikube: 1.30+

kubectrl: 1.28+

Git: 2.30+

Recommended for Smooth Operation

CPU: 4 cores

RAM: 8GB

Storage: 20GB free space

2. Complete Installation Steps

Step 1: Clone Repository

bash

git clone <https://github.com/Mathi27/end-to-end-cardio-pipeline>

cd end-to-end-cardio-pipeline

Step 2: Python Environment Setup

bash

Create virtual environment

python -m venv venv

Activate environment

On Linux/macOS:

source venv/bin/activate

On Windows:

venv\Scripts\activate

Install dependencies

pip install -r requirements.txt

Step 3: Data Preparation

bash

Create data directory

mkdir -p data/raw data/processed

Download dataset

python scripts/download_data.py

Verify data

ls -la data/raw/

Step 4: Docker Setup

bash

Build Docker image

docker build -t heart-disease-api:latest .

Test Docker image

docker run -p 5000:5000 heart-disease-api:latest

In another terminal, test API

curl http://localhost:5000/health

Step 5: Kubernetes Setup (Minikube)

bash

Start Minikube cluster

minikube start --cpus=4 --memory=8192

Enable ingress addon

minikube addons enable ingress

Set Docker environment to Minikube's daemon

eval \$(minikube docker-env)

Rebuild image for Minikube

docker build -t heart-disease-api:latest .

Step 6: Deploy Application

bash

Deploy application

kubecttl apply -f k8s/deployment.yaml

kubecttl apply -f k8s/service.yaml

Check deployment status

kubecttl get pods -w

Step 7: Deploy Monitoring Stack

bash

Create monitoring namespace

kubecttl create namespace monitoring

Deploy Prometheus

kubecttl apply -f monitoring/prometheus.yaml -n monitoring

Deploy Grafana

kubecttl apply -f monitoring/grafana.yaml -n monitoring

Check monitoring pods

kubecttl get pods -n monitoring

Step 8: Access Services

bash

Get application URL

minikube service heart-disease-service --url

Port forward Grafana

kubectrl port-forward svc/grafana-service 3000:3000 -n monitoring

Port forward Prometheus

kubectrl port-forward svc/prometheus-service 9090:9090 -n monitoring

3. Access Information

Application Endpoints

API Health: <http://localhost:5000/health>

API Prediction: <http://localhost:5000/predict>

API Metrics: <http://localhost:5000/metrics>

Monitoring Dashboards

Grafana: <http://localhost:3000> (admin/admin123)

Prometheus: <http://localhost:9090>

Kubernetes Resources

bash

View all resources

kubectrl get all

View logs

kubectl logs deployment/heart-disease-deployment -f

4. Quick Test Commands

Test API Endpoints

bash

Health check

curl http://localhost:5000/health

Make prediction

**curl -X POST http://localhost:5000/predict **

**-H "Content-Type: application/json" **

-d '{"age": 45, "sex": 1, "cp": 0, "trestbps": 120, "chol": 250, "fbs": 0, "restecg": 0, "thalach": 150, "exang": 0, "oldpeak": 1.0, "slope": 0, "ca": 1, "thal": 1}'

Check metrics

curl http://localhost:5000/metrics | head -20

Test Monitoring

bash

Check Prometheus targets

curl http://localhost:9090/targets

Verify Grafana is running

curl -I http://localhost:3000

5. Troubleshooting Guide

Common Issues & Solutions

Minikube won't start

bash

minikube delete

minikube start --cpus=4 --memory=8192 --driver=docker

Docker build fails

bash

Clean Docker cache

docker system prune -a

Rebuild with no cache

docker build --no-cache -t heart-disease-api:latest .

Pods stuck in ContainerCreating

bash

Check events

kubectl get events --sort-by='.lastTimestamp'

Describe pod

kubectl describe pod <pod-name>

Port already in use

bash

Find process using port

lsof -i :5000

Kill process

sudo kill -9 <PID>

Image pull errors

bash

Use Minikube's Docker daemon

eval \$(minikube docker-env)

docker build -t heart-disease-api:latest .

6. Cleanup Commands

Remove Kubernetes Resources

bash

Delete application

kubectl delete -f k8s/

Delete monitoring

kubectl delete -f monitoring/ -n monitoring

kubectl delete namespace monitoring

Stop Minikube

minikube stop

Remove Docker Resources

bash

Remove containers

docker stop \$(docker ps -aq)

docker rm \$(docker ps -aq)

Remove images

docker rmi heart-disease-api:latest

Clean system

docker system prune -a

7. Verification Checklist

Python virtual environment created

Dependencies installed successfully

Docker image builds without errors

Minikube cluster is running

Kubernetes pods are in "Running" state

API responds to health check

Predictions return valid JSON

Prometheus is scraping metrics

Grafana dashboard is accessible

Logs show no critical errors

8. Support Information

Log Files Location

Application logs: logs/app.log

Docker logs: docker logs <container-id>

Kubernetes logs: kubectl logs <pod-name>

Minikube logs: minikube logs

Configuration Files

Python: requirements.txt

Docker: Dockerfile

Kubernetes: k8s/ directory

Monitoring: monitoring/ directory

CI/CD: .github/workflows/ci-cd.yml

Useful Commands for Debugging

bash

Check system resources

kubectl top pods

kubectl top nodes

View detailed resource information

kubectl describe deployment heart-disease-deployment

kubectl describe service heart-disease-service

Check network connectivity

kubectl exec <pod-name> -- curl http://localhost:5000/health

View resource usage

minikube dashboard

9. Next Steps After Installation

Run EDA Notebook:

bash

jupyter notebook notebooks/EDA.ipynb

Train Model:

bash

python src/train.py

Run Tests:

bash

pytest tests/ -v

Trigger CI/CD Pipeline:

Push to GitHub repository

Check GitHub Actions tab

Monitor Application:

Access Grafana at <http://localhost:3000>

Set up alerts for error rates > 5%

Monitor prediction latency